

IF 260

Sistem Operasi

Minggu 10

Dr. Ananda Kusuma
e-mail: ananda.kusuma@lecturer.umn.ac.id

Universitas Multimedia Nusantara
Serpong, Tangerang

Agenda

- **Review:**
 - **Paging**
- **Topik Minggu 10:**
 - **Page Replacement Algorithms**

Review:

Paging & Virtual Memory

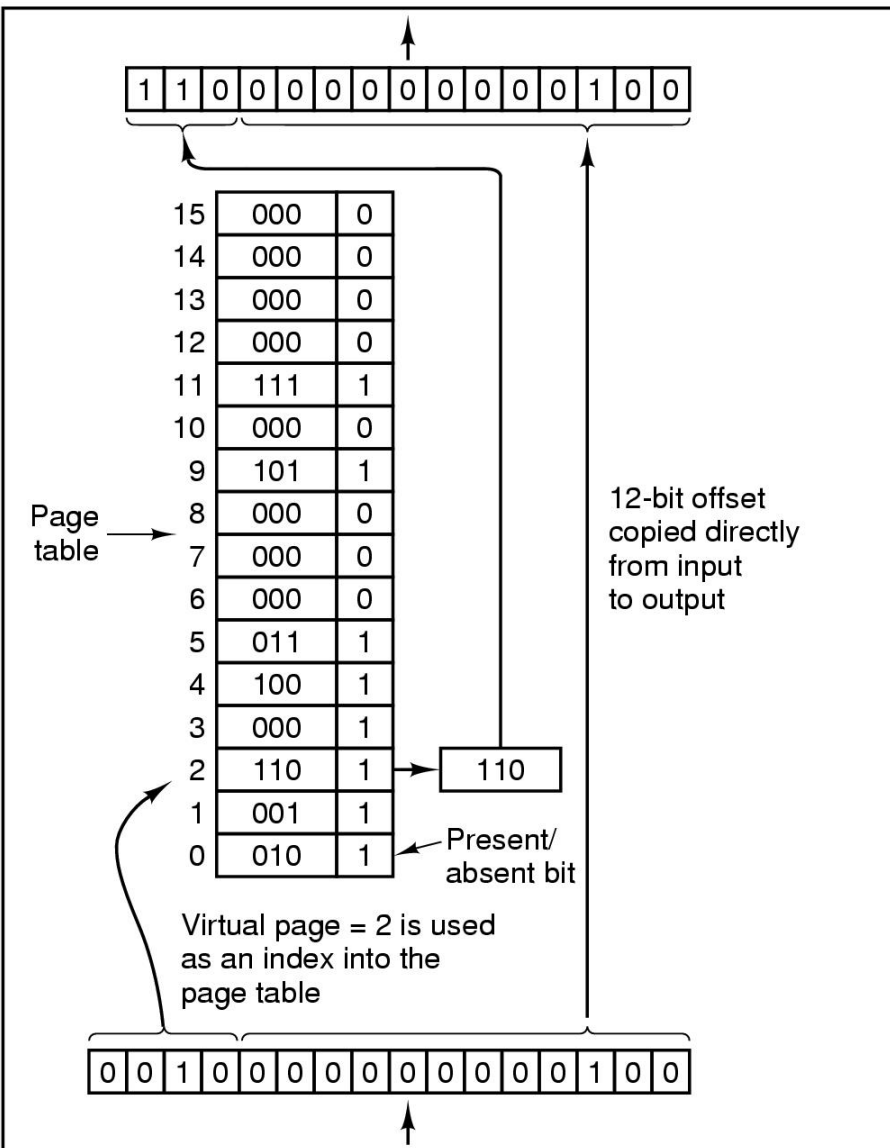
Implementasi Virtual memory dgn Paging

- Virtual address space vs Physical address space?
- Keuntungan penggunaan virtual memory?

Paging:

- Bagi virtual address space atas unit-unit kecil yang berukuran tetap yang disebut dengan Page
- Bagi physical address space atas unit-unit kecil yang disebut Page Frame, dan ukurannya sama dengan Page
- MMU melakukan pemetaan Page ke Frame, dan Frame tidak harus contiguous (berurutan) pada physical memory

Contoh generik operasi internal pada MMU untuk 16 buah 4-KB Pages



Outgoing
physical
address
(24580)

Physical address:
Page frame number & offset

Virtual page number merujuk pada
Index dari page table
Page table dikelola oleh memory
manager (kernel)
Page table entry berisikan page frame
number

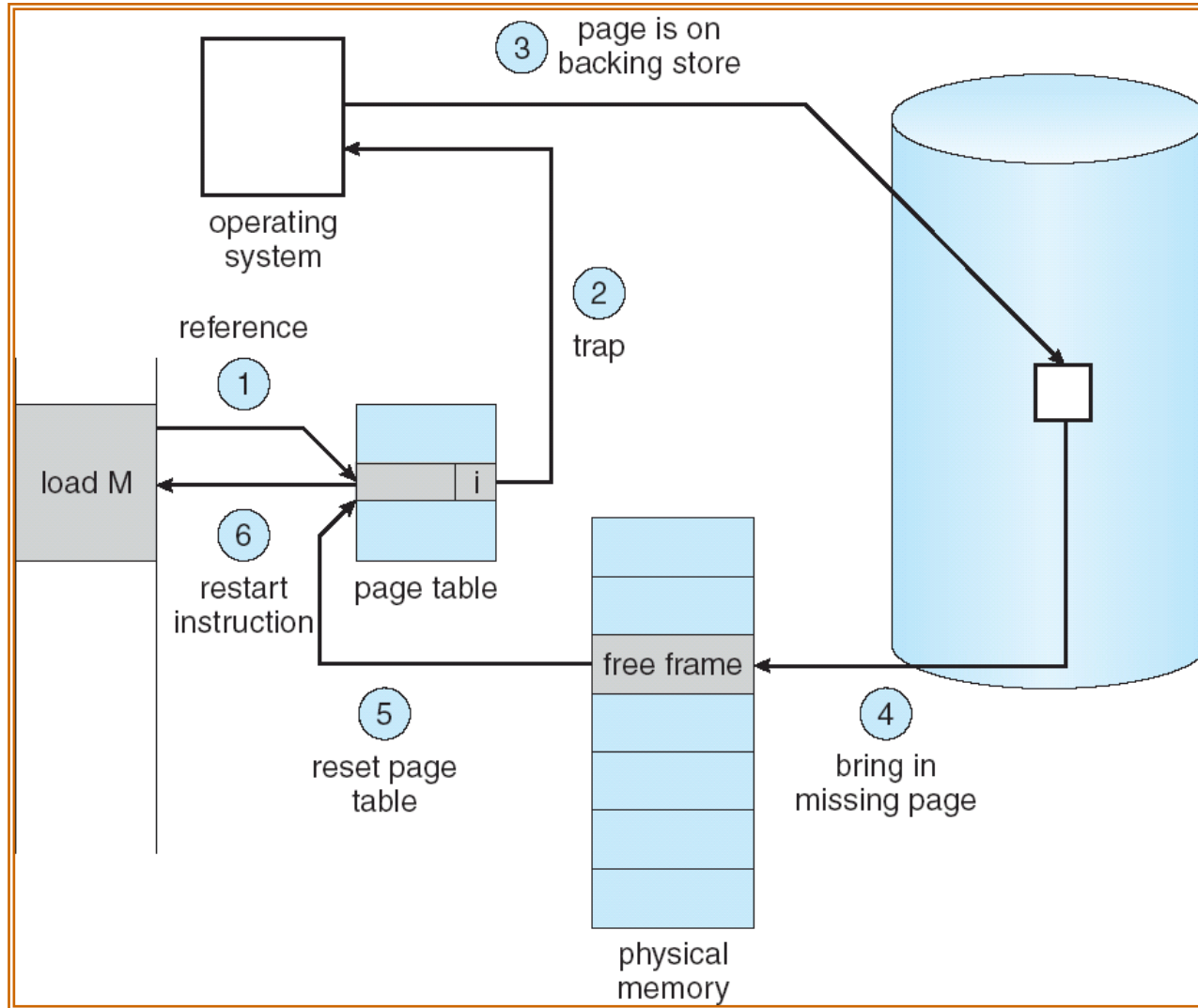
Virtual address:
Virtual page number & offset

Incoming
virtual
address
(8196)

Contoh soal page reference string

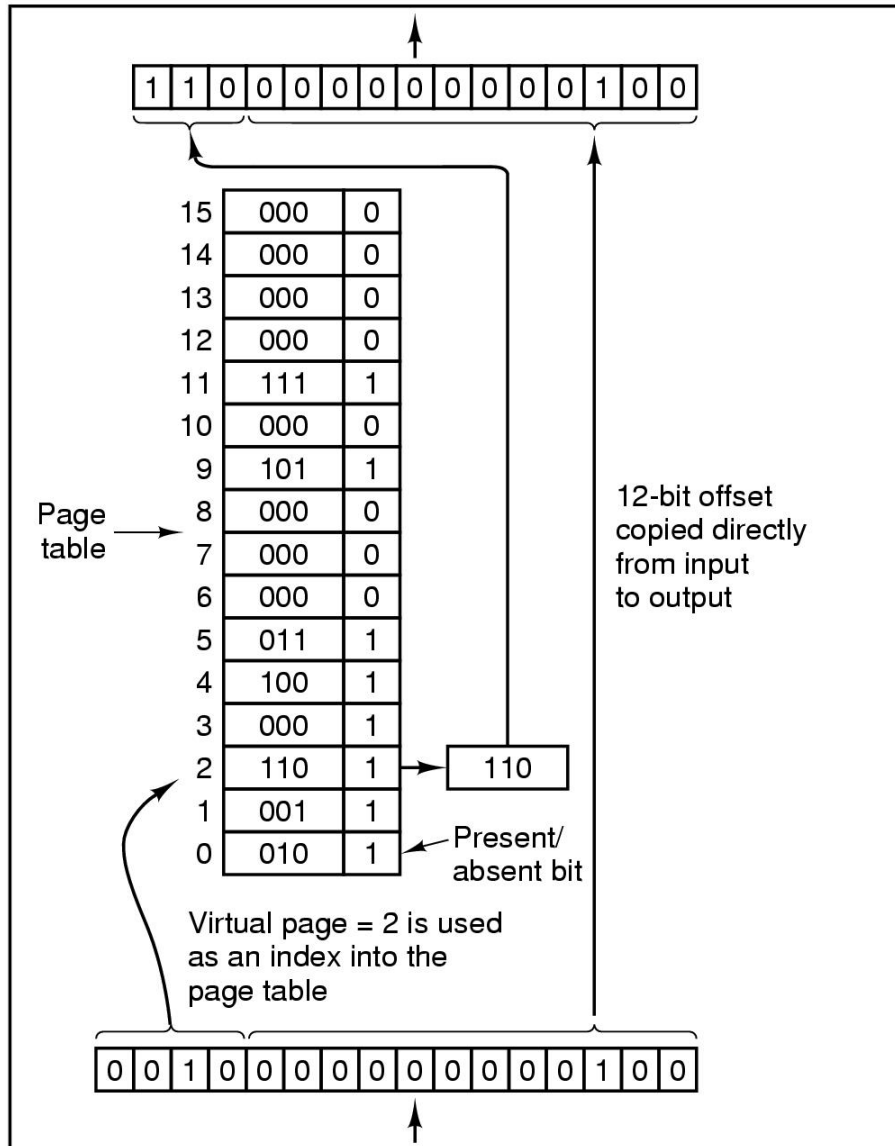
- Sebuah process secara berturut-turut mengakses alamat memory sebagai berikut: 400 401 402 1025 1020, 1027, 2049, 2050, 2602, 1028, 1542, 1546, 2051, 3585, 3603, 4097 4098, 1044, 1045.
- Ukuran tiap page adalah 512 bytes. Tiap virtual address menggunakan berapa bit offset?
- Tentukan *page reference string* dari urutan pengaksesan memori tersebut.

Apabila terjadi Page Fault



Page Replacement Algorithms

Contoh Paging dengan 16 buah 4-KB Pages



Outgoing
physical
address
(24580)

Reference string: urutan akses page
sebuah process

Contoh:

0 1 2 3 4 5

0 2 7 3 4 5

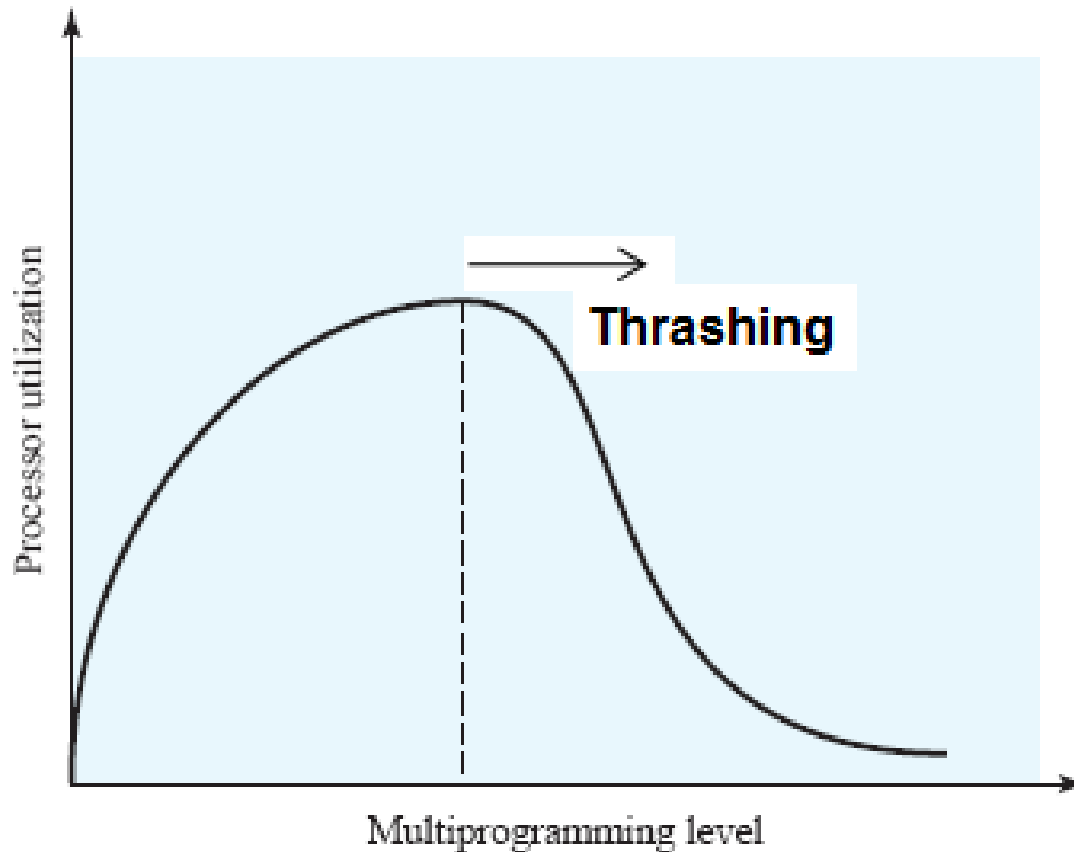
2 3 0 8 4 5

Incoming
virtual
address
(8196)

Page Replacement Algorithm

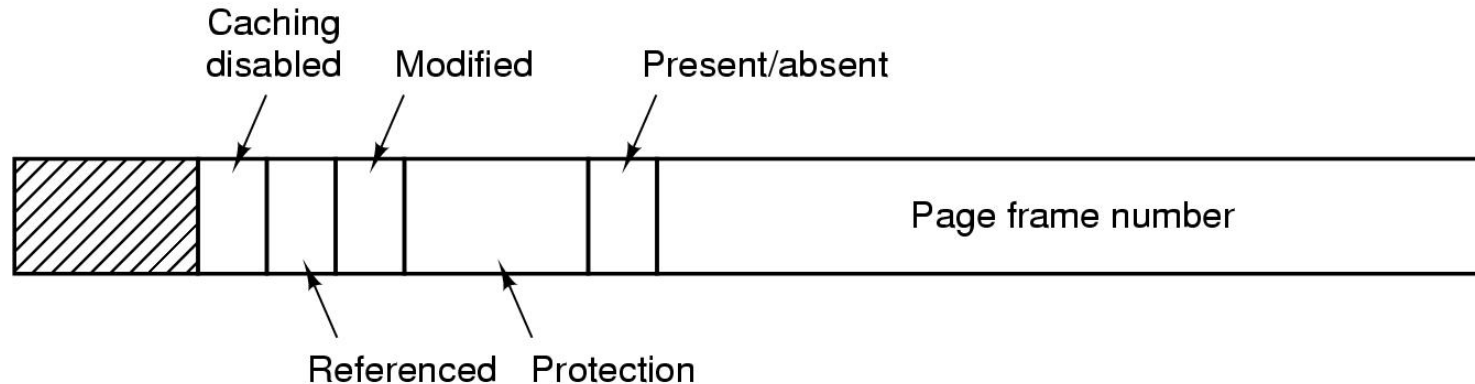
- **Jika ada page-fault untuk penggantian page, OS harus memilih page untuk dikeluarkan dari memory, sehingga ada tempat untuk page yang baru diminta**
 - **Jika page yang dikeluarkan berstatus modified (dirty bit = 1), maka harus ditulis ulang ke disk**
 - **Page yang dikeluarkan dari memory dapat merupakan page dari process yang menyebabkan page-fault, ataupun page dari process lain**
- **Kinerja sistem bagus apabila page-fault rendah**
 - **Pilih page replacement yang menghasilkan page-fault terendah**
 - **Trade-off optimality vs complexity**
 - **Evaluasi dengan menjalankan string-string tertentu dari memory reference, dan hitung jumlah page fault.**

Dampak Thrashing



Process menghabiskan waktu lebih banyak untuk paging daripada eksekusi (sibuk melakukan swap-in swap-out pages)

Struktur generik Page Table Entry (PTE)



- **Page frame number:** menunjukkan physical frame di mana address dimulai
- **Present/absent:** apakah page yang dirujuk ada di memory
- **Protection:** operasi (read, write, execute) yang diperbolehkan
- **Modified / dirty bit/ M-bit:** apakah page mengalami modifikasi
 - Apabila ada modifikasi, maka konten memory harus ditulis ulang ke disk saat ada pergantian page (page replacement)
- **Referenced / R-Bit:** apakah page sedang dirujuk (memory reference) oleh program
- **Caching disabled:** untuk page yang dipetakan ke I/O device register

Detil Implementasi PTE: Tergantung Arsitektur

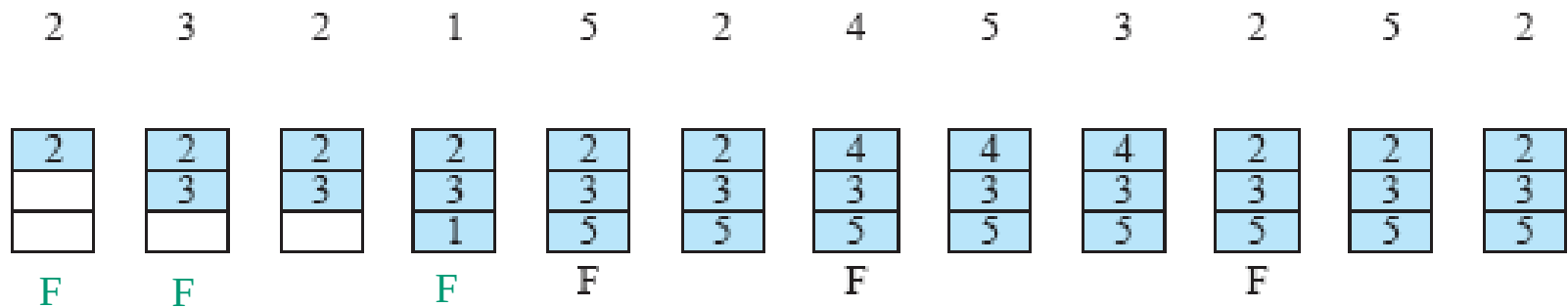
Contoh diambil dari file pgtable.h (pada Linux kernel source) untuk arsitektur X86

```
...
#define _PAGE_BIT_PRESENT      0      /* is present */
#define _PAGE_BIT_RW          1      /* writeable */
#define _PAGE_BIT_USER        2      /* userspace addressable */
#define _PAGE_BIT_PWT         3      /* page write through */
#define _PAGE_BIT_PCD         4      /* page cache disabled */
#define _PAGE_BIT_ACCESSED    5      /* was accessed (raised by CPU) */
#define _PAGE_BIT_DIRTY       6      /* was written to (raised by CPU) */
#define _PAGE_BIT_FILE        6
#define _PAGE_BIT_PSE         7      /* 4 MB (or 2MB) page */
#define _PAGE_BIT_PAT         7      /* on 4KB pages */
#define _PAGE_BIT_GLOBAL      8      /* Global TLB entry PPro+ */
#define _PAGE_BIT_UNUSED1     9      /* available for programmer */
#define _PAGE_BIT_IOMAP       10     /* flag used to indicate IO mapping */
#define _PAGE_BIT_UNUSED3     11
#define _PAGE_BIT_PAT_LARGE   12     /* On 2MB or 1GB pages */
....
```

Optimal Page Replacement Algorithm (Belady's Algorithm)

- Memilih page yang nantinya akan diacu paling terakhir untuk dikeluarkan
 - Optimal (**page fault paling rendah**), tetapi tidak realistik karena mensyaratkan OS punya informasi sempurna akan event-event yang akan terjadi di depan

- Contoh:



- Total page-fault = 6
- Jumlah page-fault untuk penggantian page = 3
- Untuk slide2x selanjutnya, fokus pada page-fault untuk penggantian page untuk membandingkan page replacement algorithm

Not Recently Used (NRU) Page Replacement Algorithm

- Menggunakan informasi status bit: R (Reference) dan M (Modified) bit pada page table entry
- Page dimasukkan ke kelompok dengan klasifikasi sbb.:
 - Class 0: not referenced, not modified
 - Class 1: not referenced, modified
 - Class 2: referenced, not modified
 - Class 3: referenced, modified
- Penggantian secara random (acak) page dari class dengan nomor terendah

First-In, First-Out (FIFO) Page Replacement Algorithm

- Page frames yang lebih dahulu di-loaded ke memory yang akan lebih dahulu dikeluarkan apabila ada page-fault
 - Keuntungan: sederhana untuk diimplementasikan
 - Kekurangan: asumsi bahwa page yang paling lama berada di memory akan paling kecil peluangnya untuk diacu sering kali salah → kinerja algoritma ini kurang bagus

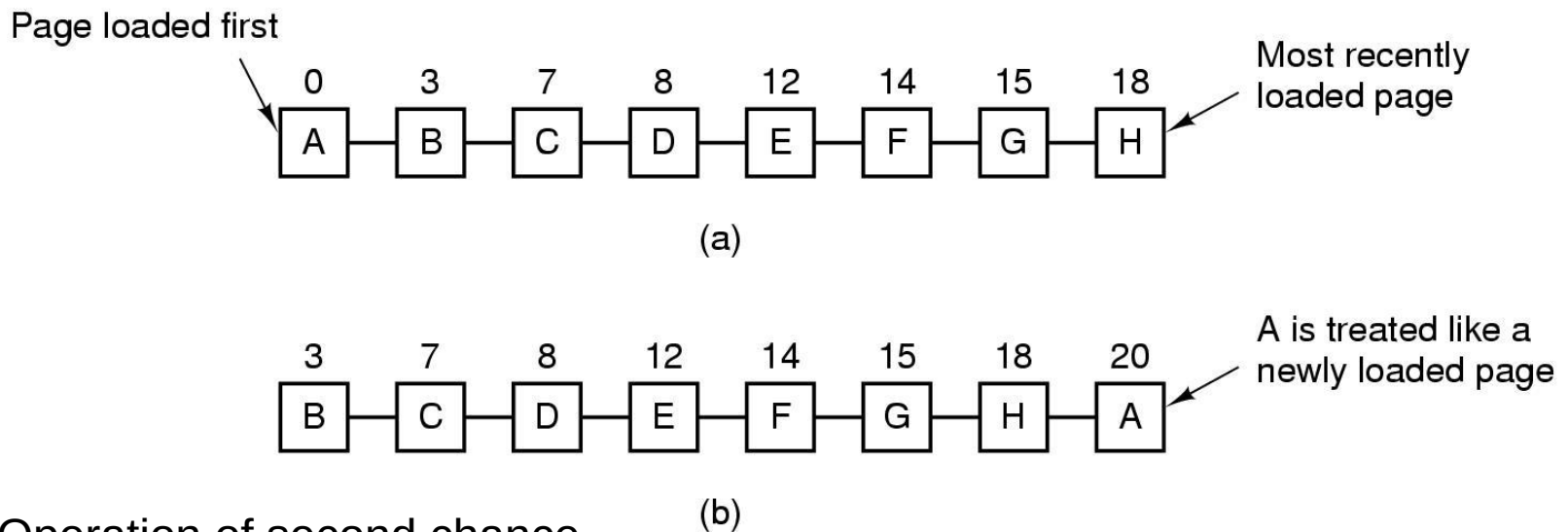
- Contoh:

2	3	2	1	5	2	4	5	3	2	5	2																																				
<table><tr><td>2</td></tr><tr><td></td></tr><tr><td></td></tr></table>	2			<table><tr><td>2</td></tr><tr><td>3</td></tr><tr><td></td></tr></table>	2	3		<table><tr><td>2</td></tr><tr><td>3</td></tr><tr><td></td></tr></table>	2	3		<table><tr><td>2</td></tr><tr><td>3</td></tr><tr><td>1</td></tr></table>	2	3	1	<table><tr><td>5</td></tr><tr><td>3</td></tr><tr><td>1</td></tr></table> F	5	3	1	<table><tr><td>5</td></tr><tr><td>2</td></tr><tr><td>1</td></tr></table> F	5	2	1	<table><tr><td>5</td></tr><tr><td>2</td></tr><tr><td>4</td></tr></table> F	5	2	4	<table><tr><td>5</td></tr><tr><td>2</td></tr><tr><td>4</td></tr></table>	5	2	4	<table><tr><td>3</td></tr><tr><td>2</td></tr><tr><td>4</td></tr></table> F	3	2	4	<table><tr><td>3</td></tr><tr><td>2</td></tr><tr><td>4</td></tr></table>	3	2	4	<table><tr><td>3</td></tr><tr><td>5</td></tr><tr><td>4</td></tr></table> F	3	5	4	<table><tr><td>3</td></tr><tr><td>5</td></tr><tr><td>2</td></tr></table> F	3	5	2
2																																															
2																																															
3																																															
2																																															
3																																															
2																																															
3																																															
1																																															
5																																															
3																																															
1																																															
5																																															
2																																															
1																																															
5																																															
2																																															
4																																															
5																																															
2																																															
4																																															
3																																															
2																																															
4																																															
3																																															
2																																															
4																																															
3																																															
5																																															
4																																															
3																																															
5																																															
2																																															

- Jumlah page-fault untuk penggantian page = 6

Second-Chance Page Replacement Algorithm

- Modifikasi dari FIFO dengan menambahkan informasi R bit (1 atau 0) untuk menunjukkan apakah page diacu pada interval clock yang terbaru
 - Jika terkena gilirannya berdasarkan FIFO, set $R=0$, biarkan di memory dan berikan kesempatan kedua, dan page tersebut dipindahkan ke ujung belakang dari link-list



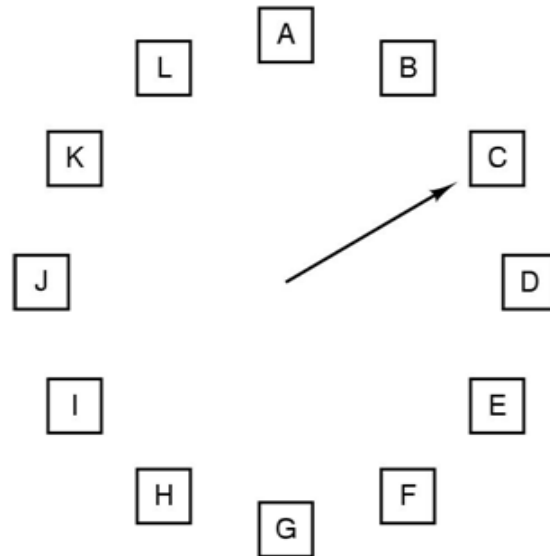
Operation of second chance.

(a) Pages sorted in FIFO order.

(b) Page list if a page fault occurs at time 20 and A has its R bit set. The numbers above the pages are their load times.

Clock Page Replacement Algorithm

- Penyempurnaan dari second-change, dengan cara menetapkan page frame dalam bentuk circular-list, dan ada pointer yang menunjuk ke page yang tertua



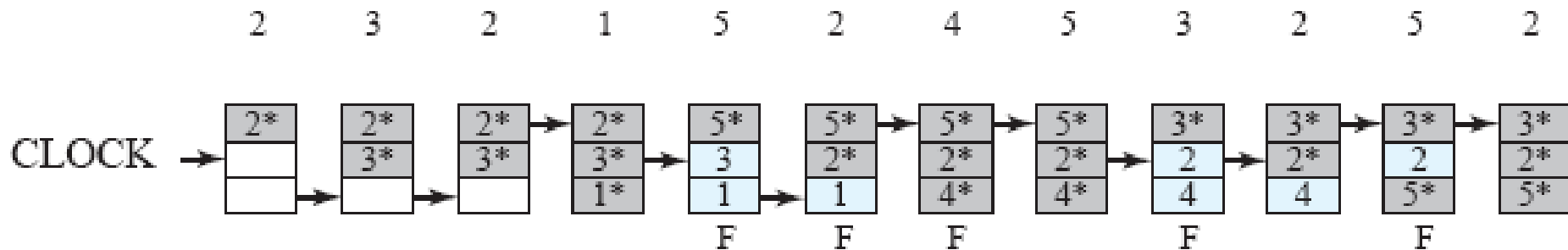
When a page fault occurs,
the page the hand is
pointing to is inspected.
The action taken depends
on the R bit:

R = 0: Evict the page

R = 1: Clear R and advance hand

Clock Page Replacement Algorithm: Contoh

- Contoh:



- Bagaimana gambaran clock kalau mengacu ke slide sebelumnya?
- Jumlah page fault untuk penggantian page = 5

Least-Recently Used (LRU) Page Replacement Algorithm

- Keluarkan page yang paling lama tidak terpakai
- Mendekati optimal algorithm, namun implementasi lebih rumit dan lebih memakan waktu
 - Mengatur pengurutan page pada link-list setiap memory reference
 - Counter untuk tiap page yang di-update secara otomatis untuk setiap memory reference
- Contoh:

2	3	2	1	5	2	4	5	3	2	5	2																																				
<table><tr><td>2</td></tr><tr><td></td></tr><tr><td></td></tr></table>	2			<table><tr><td>2</td></tr><tr><td>3</td></tr><tr><td></td></tr></table>	2	3		<table><tr><td>2</td></tr><tr><td>3</td></tr><tr><td></td></tr></table>	2	3		<table><tr><td>2</td></tr><tr><td>3</td></tr><tr><td>1</td></tr></table>	2	3	1	<table><tr><td>2</td></tr><tr><td>5</td></tr><tr><td>1</td></tr></table>	2	5	1	<table><tr><td>2</td></tr><tr><td>5</td></tr><tr><td>1</td></tr></table>	2	5	1	<table><tr><td>2</td></tr><tr><td>5</td></tr><tr><td>4</td></tr></table>	2	5	4	<table><tr><td>2</td></tr><tr><td>5</td></tr><tr><td>4</td></tr></table>	2	5	4	<table><tr><td>3</td></tr><tr><td>5</td></tr><tr><td>4</td></tr></table>	3	5	4	<table><tr><td>3</td></tr><tr><td>5</td></tr><tr><td>2</td></tr></table>	3	5	2	<table><tr><td>3</td></tr><tr><td>5</td></tr><tr><td>2</td></tr></table>	3	5	2	<table><tr><td>3</td></tr><tr><td>5</td></tr><tr><td>2</td></tr></table>	3	5	2
2																																															
2																																															
3																																															
2																																															
3																																															
2																																															
3																																															
1																																															
2																																															
5																																															
1																																															
2																																															
5																																															
1																																															
2																																															
5																																															
4																																															
2																																															
5																																															
4																																															
3																																															
5																																															
4																																															
3																																															
5																																															
2																																															
3																																															
5																																															
2																																															
3																																															
5																																															
2																																															
				F		F		F	F																																						

- Jumlah page-fault untuk penggantian page = 4

Contoh Soal:

- 2) Suatu sistem komputer memiliki 4 page frame. Informasi waktu saat di-loaded ke memory, waktu terakhir diakses dan informasi R dan M bit diberikan pada tabel berikut:

Page	Loaded	Last Ref	R	M
0	126	280	1	0
1	230	265	0	1
2	140	270	0	0
3	110	285	1	1

Tentukan page yang akan diganti apabila menggunakan

- a) FIFO
- b) NRU
- c) LRU
- d) Second chance

Contoh Soal:

1) Diketahui sebuah reference string berikut: " 1 2 1 7 6 7 3 4 3 5 6 7 ".

Jika process mendapat alokasi tiga page frame, gambarkan pemanfaatan frame tersebut dan tentukan jumlah page fault untuk penggantian page menggunakan algoritma:

- a) Optimal Page Replacement**
- b) FIFO**
- c) Clock Page Replacement**
- d) Least-Recently Used (LRU)**

LRU dengan Hardware Counter

- Nilai counter secara otomatis di-update untuk memory reference. Page table entry punya field cukup besar untuk menampung nilai counter
- Contoh: sistem dengan 4 page frame dan LRU hardware yang mengelola 4x4 matrix. Tiap kali page k di-referenced, baris k = 1, kolom k = 0. Baris dengan nilai binari terendah adalah “the LRU page”
- Page reference string: 0 1 2 3 2 1 0 3 2 3

Page				
	0	1	2	3
0	0	1	1	1
1	0	0	0	0
2	0	0	0	0
3	0	0	0	0

(a)

Page				
	0	1	2	3
0	0	0	1	1
1	1	0	1	1
2	0	0	0	0
3	0	0	0	0

(b)

Page				
	0	1	2	3
0	0	0	0	1
1	1	0	0	1
2	1	1	0	1
3	0	0	0	0

(c)

Page				
	0	1	2	3
0	0	0	0	0
1	1	0	0	0
2	1	1	0	0
3	1	1	1	0

(d)

Page				
	0	1	2	3
0	0	0	0	0
1	1	0	0	0
2	1	1	0	1
3	1	1	0	0

(e)

0	0	0	0
1	0	1	1
1	0	0	1
1	0	0	0

(f)

0	1	1	1
0	0	1	1
0	0	0	1
0	0	0	0

(g)

0	1	1	0
0	0	1	0
0	0	0	0
1	1	1	0

(h)

0	1	0	0
0	0	0	0
1	1	0	1
1	1	0	0

(i)

0	1	0	0
0	0	0	0
1	1	0	0
1	1	1	0

(j)

Aging Algorithm:

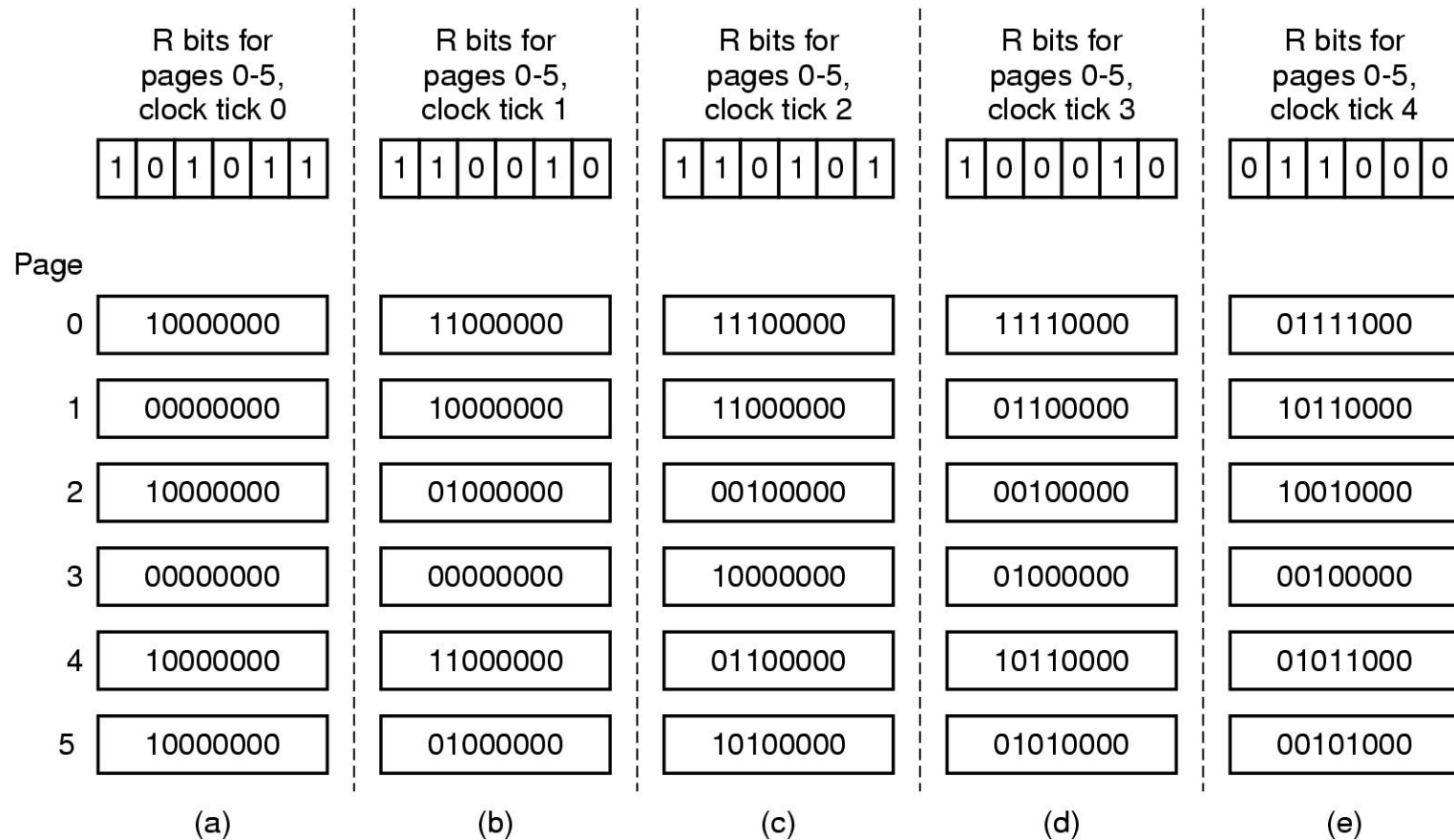
Simulasi LRU dengan Software Counter

- **Tidak semua sistem memiliki LRU counter hardware**
 - **Pendekatan dengan software counter**
- **Buat counter untuk tiap Page Table Entry**
- **Nilai counter menandakan jumlah interval kapan page terakhir diakses**
- **Keluarkan page yang memiliki nilai counter terendah**
- **Secara periodik (dengan clock interrupt) periksa R bit:**
 - **NFU (Not Frequently Used): nilai R ditambahkan ke counter**
 - **Problem jika fase awal dari eksekusi punya waktu eksekusi jauh lebih lama dibandingkan keseluruhan fase**
 - **Aging Algorithm:**
 - **Tiap clock tick, nilai counter digeser 1 bit ke kanan**
 - **Bit paling kiri pada counter ditambahkan dengan R bit**

Aging Algorithm: Simulasi LRU dengan Software Counter

- **Buat counter untuk tiap Page Table Entry**
- **Secara periodik (dengan clock interrupt) periksa R bit:**
 - **Jika $R = 0$, nilai counter digeser 1 bit ke kanan**
 - **Jika $R = 1$, bit paling kiri pada counter = 1**
- **Nilai counter menandakan jumlah interval kapan page terakhir diakses**
- **Keluarkan page yang memiliki nilai counter terendah**

Aging Algorithm: Simulasi LRU dengan Software Counter



6 pages dan 5 clock ticks. Kelima clock ticks ditunjukkan oleh (a) - (e).

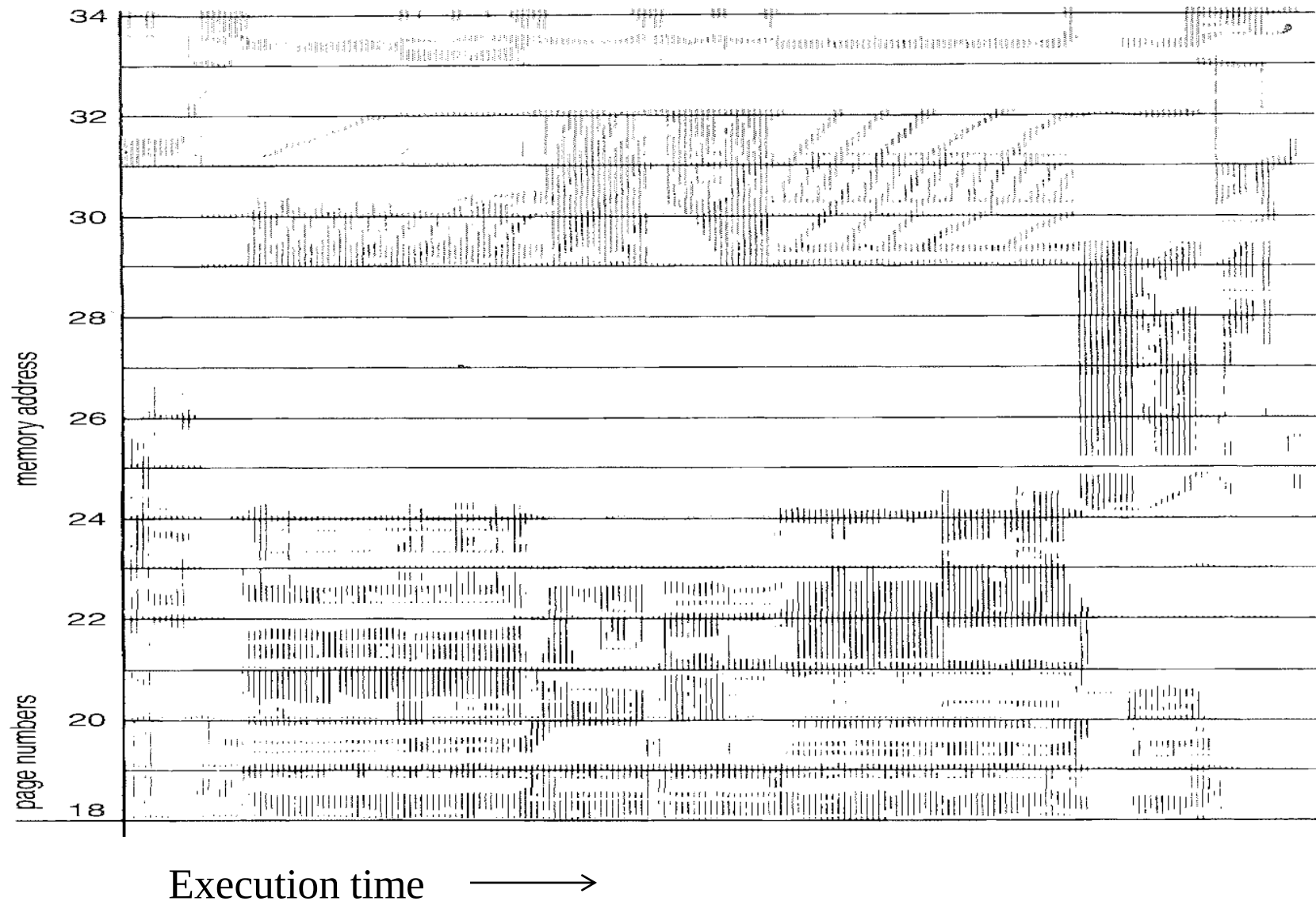
- Jika ada page-fault, keluarkan page dengan nilai software counter terendah

Page Replacement Algorithm: Working Set

Working Set

- **Locality reference (Prinsip lokalitas) → observasi menunjukkan memory references cenderung tak seragam (bersifat lokal)**
 - **Temporal locality**
 - Contoh: looping, subroutine, stack, dsb.
 - **Spatial locality**
 - Contoh: traversal pada array, eksekusi code secara sequential, kecenderungan programmer menempatkan variable yang terkait saling berdekatan, dsb.
- **Konsekuensinya, program dapat berjalan secara lebih efisien saat satu himpunan bagian dari page-page berkecenderungan tinggi untuk diacu terdapat di memory**
- **Working set (himpunan kerja) adalah himpunan page-page dari suatu process yang secara aktif diacu.**
 - Oleh karena itu, himpunan ini harus dijaga untuk berada di physical memory agar program berjalan secara efisien

Locality pada suatu pola memory reference

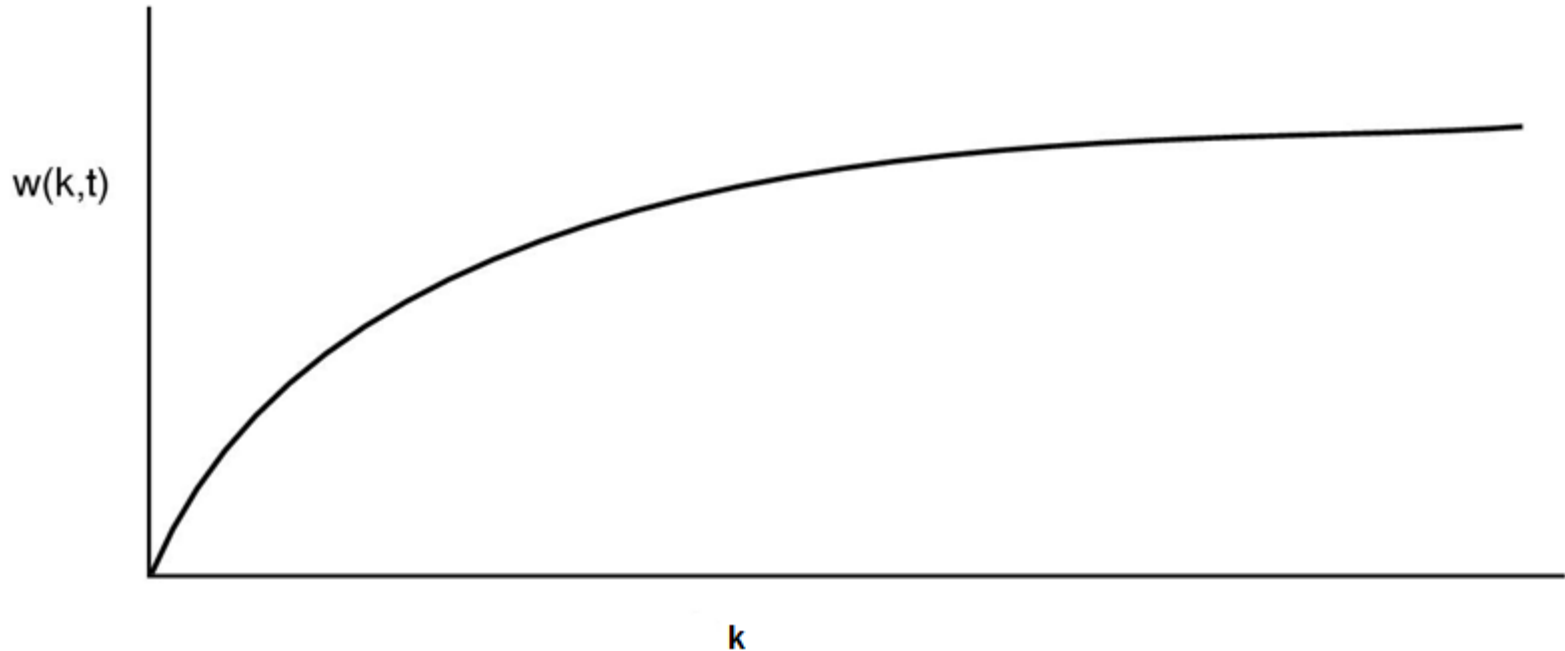


Working Set: Contoh

Sequence of Page References	Window Size, Δ			
	2	3	4	5
24	24	24	24	24
15	24 15	24 15	24 15	24 15
18	15 18	24 15 18	24 15 18	24 15 18
23	18 23	15 18 23	24 15 18 23	24 15 18 23
24	23 24	18 23 24	•	•
17	24 17	23 24 17	18 23 24 17	15 18 23 24 17
18	17 18	24 17 18	•	18 23 24 17
24	18 24	•	24 17 18	•
18	•	18 24	•	24 17 18
17	18 17	24 18 17	•	•
17	17	18 17	•	•
15	17 15	17 15	18 17 15	24 18 17 15
24	15 24	17 15 24	17 15 24	•
17	24 17	•	•	17 15 24
24	•	24 17	•	•
18	24 18	17 24 18	17 24 18	15 17 24 18

Working Set Function

- Working set $w(k,t)$: himpunan yang berisikan semua pages yang digunakan oleh k most recent memory references
- Dampak lokalitas ditunjukkan pada fungsi di bawah



Prinsip-prinsip Working Set Model (1)

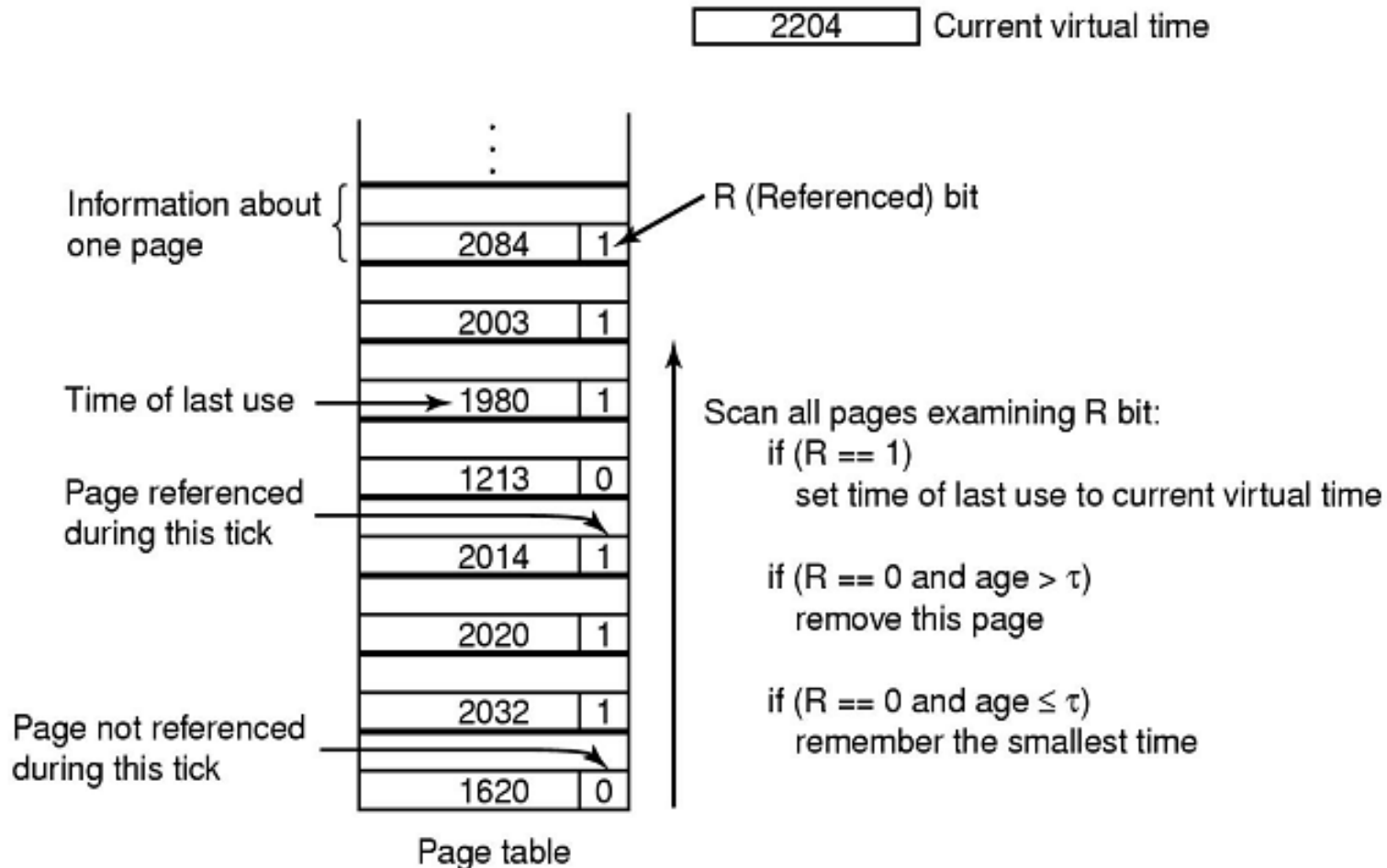
- **Program seharusnya dijalankan jika dan hanya jika working set berada di memory**
- **Dua teknik untuk memuatkan page**
 - **Prepaging: teknik memuatkan page-page sebelum process berjalan**
 - **Demand paging: teknik yang segera memuatkan page begitu page dibutuhkan**

Prinsip-prinsip Working Set Model (2)

- Cara menentukan working set
 - Menghitung ke belakang k memory references sangat mahal, sebagai gantinya gunakan waktu eksekusi
 - Contoh: page-page yang diacu pada 100ms waktu eksekusi ke belakang
 - Waktu eksekusi CPU ditentukan per process $\rightarrow$ virtual time
 - Time-stamp (virtual time) untuk tiap page reference
 - Anggota dari working set adalah page-page yang age (usianya) kurang dari window size (ukuran jendela) atau interval waktu dari Working Set yang ditetapkan:
 - Gunakan timer dan reference bit
 - Tentukan timer untuk interrupt tiap waktu τ (kira-kira seukuran window size)
 - Keluarkan dari working set page-page yang tidak di-referenced dan reset reference bit

$$\text{Age} = \text{current_virtual_time} - \text{time_of_last_used}$$

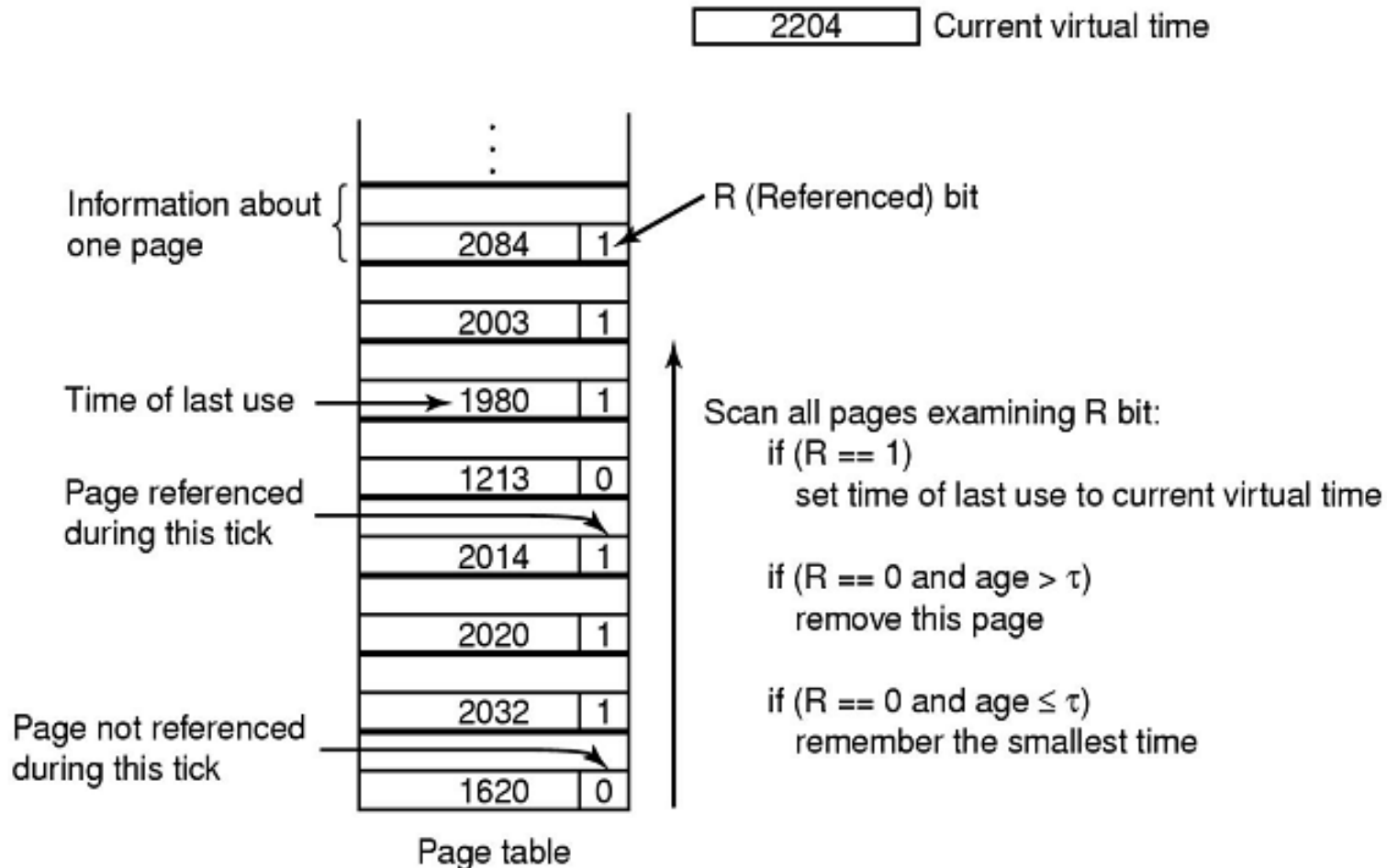
Working Set Page Replacement Algorithm



Age = currentvirtual time – time of last use

Ukuran jendela τ ditentukan secara empiris. Jika terlalu pendek, tidak sepenuhnya memuat lokalitas saat itu. Jika terlalu panjang, memuat page frames yang tidak diperlukan

Working Set Page Replacement Algorithm: Contoh



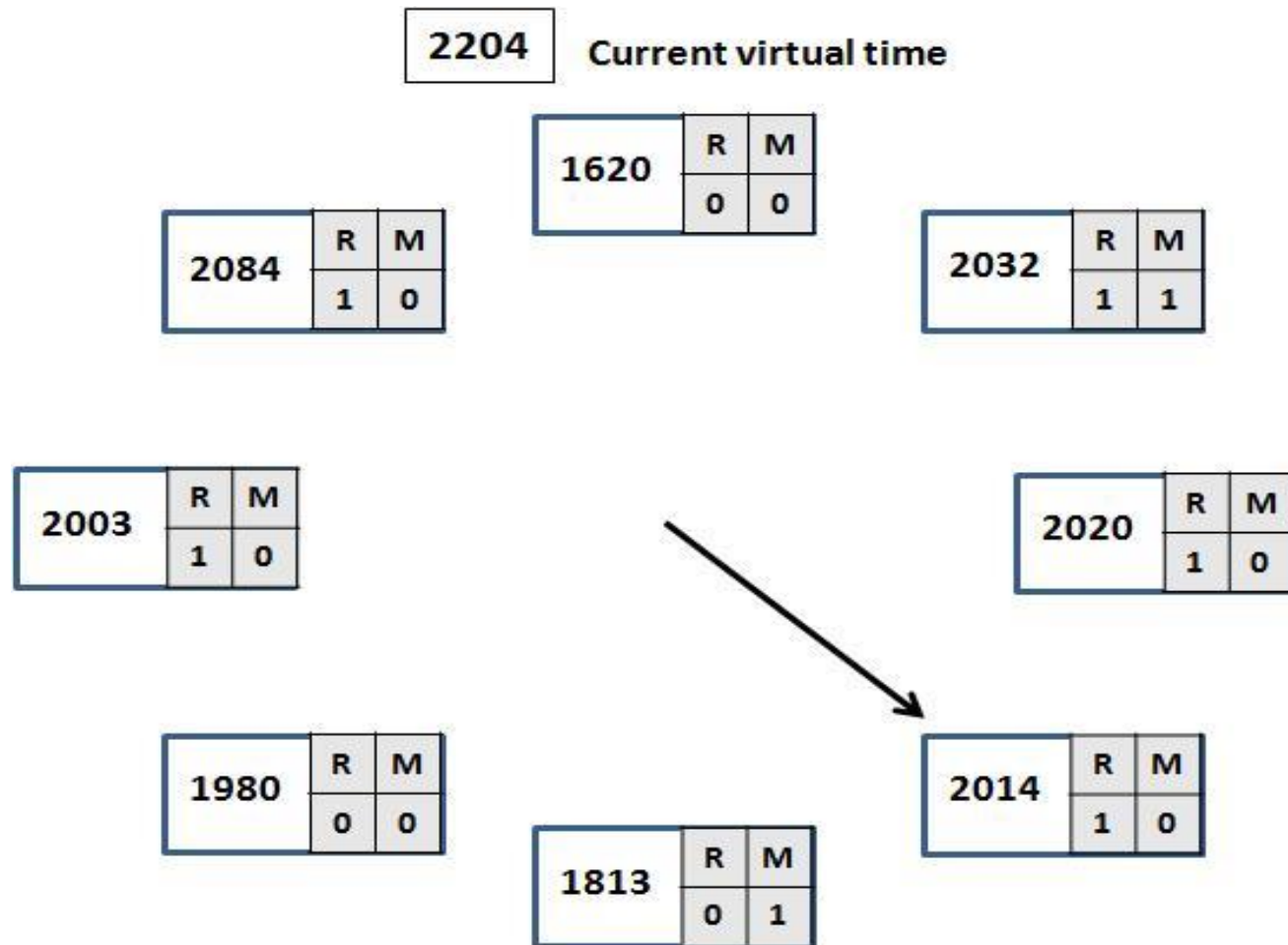
Jika window size $\tau=600$

Tentukan page yang akan dikeluarkan

WSClock Page Replacement Algorithm

- **Struktur data:** circular list of page frame (tidak perlu scan keseluruhan isi page table)
 - Informasi berisikan R (reference bit), M (modified: dirty/clean bit), dan time of last used (time-stamp)
 - R dan time-stamp di-update tiap clock tick
- **Clock bergerak dengan arah tertentu mengitari list. Untuk tiap entry:**
 - Jika $R = 1$, reset R dan update time → second chance
 - Jika $R = 0$, dan jika $age < Window_Size$, lanjutkan karena masih anggota working set → third chance
 - Jika $R = 0$, dan jika $age > Window_Size$, dan jika dirty, maka jadwalkan penulisan isi page ke disk → fourth chance
 - Jika $R = 0$, dan jika $age > Window_Size$, dan jika clean, maka page bisa dikeluarkan
- Lanjutkan ke entry berikutnya

WSClock Page Replacement Algorithm: Contoh



Diketahui ukuran working set (window size) $\Delta = 200$.

Tentukan page-page yang akan diganti, jika terjadi page fault saat virtual time = 2204 dan 2205

Ringkasan:

Page Replacement Algorithm

Algorithm	Comment
Optimal	Not implementable, but useful as a benchmark
NRU (Not Recently Used)	Very crude approximation of LRU
FIFO (First-In, First-Out)	Might throw out important pages
Second chance	Big improvement over FIFO
Clock	Realistic
LRU (Least Recently Used)	Excellent, but difficult to implement exactly
NFU (Not Frequently Used)	Fairly crude approximation to LRU
Aging	Efficient algorithm that approximates LRU well
Working set	Somewhat expensive to implement
WSClock	Good efficient algorithm

Design Issue: page size

- Pilih besar atau kecil?
- Dengan mempertimbangkan hanya page table size dan internal fragmentation, suatu pendekatan:

$$\text{Overhead} = se/p + p/2$$

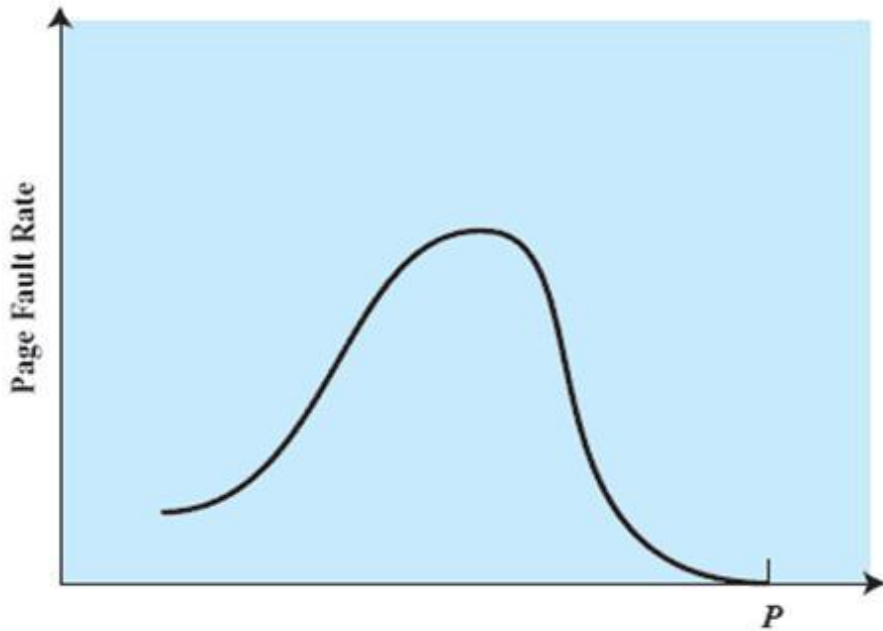
s : average process size

e : page table entry size

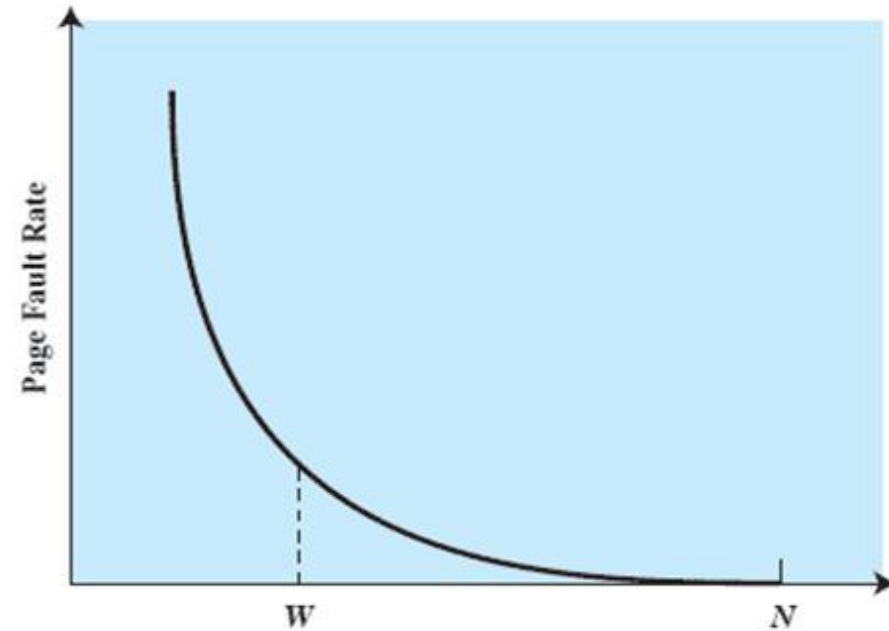
p : page size

- Optimum page size: $p^* = \sqrt{2se}$

Page size



(a) Page Size



(b) Number of Page Frames Allocated

P = size of entire process

W = working set size

N = total number of pages in process

Akhir Kuliah Minggu 10

Terima kasih atas perhatiannya!